

Lab: 2D Convolution -- AXI-Stream & AXI-Lite Array

This notebook demonstrates how to interface with an HLS 2D Convolution IP. It showcases:

1. **AXI-Stream** via DMA for high-speed image data transfer.
2. **AXI-Lite** for scalar registers (rows , cols).
3. **AXI-Lite Memory Map** for passing a 3x3 kernel array.

1. Load Overlay

```
In [1]: import time
import struct
import numpy as np
import matplotlib.pyplot as plt
from pyng import Overlay, allocate
from scipy.signal import convolve2d

ol = Overlay("design_1.bit")
print("Overlay loaded")
print("IP blocks:", list(ol.ip_dict.keys()))

dma      = ol.axi_dma_0
conv2d    = ol.conv2d_stream_0
```

```
Overlay loaded
IP blocks: ['conv2d_stream_0', 'axi_timer_0', 'axi_dma_0', 'process
ing_system7_0']
```

2. Prepare Test Image & Kernel

We will generate a simple synthetic image and use a standard 3x3 Edge Detection kernel.

Note: MAX_WIDTH is set to 64 in the HLS code, so cols must be ≤ 64 .

```
In [2]: ROWS = 64
COLS = 64
import cv2
img=cv2.imread("cat.jpg")
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
gray_resized = cv2.resize(gray,(64,64))
image_in = gray_resized.astype(np.float32)/255.0

# 3x3 Edge Detection Kernel
# kernel_5x5 = np.array([
#     [-1, -1, -1, -1, -1],
#     [-1, -1, -1, -1, -1],
#     [-1, -1, 8, -1, -1],
#     [-1, -1, -1, -1, -1],
#     [-1, -1, -1, -1, -1]
# ], dtype=np.float32)
kernel_5x5 = np.array([
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1]
], dtype=np.float32)

print(f"Image size: {ROWS}x{COLS}, Kernel:\n{kernel_5x5}")
```

Image size: 64x64, Kernel:

```
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
```

3. Configure IP (AXI-Lite)

We must write `rows` and `cols` to their respective scalar registers. For the `kernel` array, Vitis HLS maps it to a memory block on the AXI-Lite interface. We must pack Python floats into 32-bit unsigned integers and write them sequentially.

(Check `xconv2d_stream_hw.h` in your Vitis HLS export to confirm these exact offsets)

```
In [3]: CTRL_REG      = 0x00
        ROWS_OFFSET   = 0x10
        COLS_OFFSET   = 0x18
        KERNEL_BASE   = 0x80

        def float_to_uint(f):
            return struct.unpack('<I', struct.pack('<f', f))[0]

        # 1. Write dimensions
        conv2d.write(ROWS_OFFSET, ROWS)
        conv2d.write(COLS_OFFSET, COLS)

        kernel_flat = kernel_5x5.flatten()
        for i, val in enumerate(kernel_flat):
            addr = KERNEL_BASE + (i * 4)
            conv2d.write(addr, float_to_uint(val))
```

4. Execute DMA Transfer

Data path: PS → DMA TX → conv2d IP → DMA RX → PS

In [4]:

```

from pynq import allocate
import numpy as np
import time

# — Sanity checks —
assert image_in.shape == (ROWS, COLS), \
    f"Image shape mismatch: expected ({ROWS},{COLS}), got {image_in.s
assert image_in.dtype == np.float32, \
    f"Expected float32, got {image_in.dtype}"

# — Allocate DMA buffers —
in_buf = allocate(shape=(ROWS * COLS,), dtype=np.float32)
out_buf = allocate(shape=(ROWS * COLS,), dtype=np.float32)

np.copyto(in_buf, image_in.flatten())
out_buf[:] = 0

# — Correctly reset: stop then START both channels —
dma.sendchannel.stop()
dma.recvchannel.stop()
dma.sendchannel.start()
dma.recvchannel.start()

# — Start IP BEFORE DMA —
CTRL_REG = 0x00
AP_IDLE = 0x04
AP_START = 0x01

status = conv2d.read(CTRL_REG)
assert status & AP_IDLE, f"IP not idle! CTRL={status:#04x}"
conv2d.write(CTRL_REG, AP_START)

# — DMA transfer —
t0 = time.perf_counter()

dma.recvchannel.transfer(out_buf)    # recv armed first
dma.sendchannel.transfer(in_buf)

dma.sendchannel.wait()
dma.recvchannel.wait()

t_dma = time.perf_counter() - t0
print(f"Processed {ROWS*COLS} pixels in {t_dma*1e3:.2f} ms")

# — Reshape output —
image_out_hw = np.array(out_buf).reshape((ROWS, COLS))

```

Processed 4096 pixels in 2.80 ms

5. Verification & Visualization

The HLS architecture reads the input stream but doesn't output valid calculated pixels until the line buffers fill up. Because of this lack of border padding, the outermost 1-pixel border will be invalid. We crop `[1:-1, 1:-1]` to compare the valid processing window.

```
In [5]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_5x5, mode='same', boundary

hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

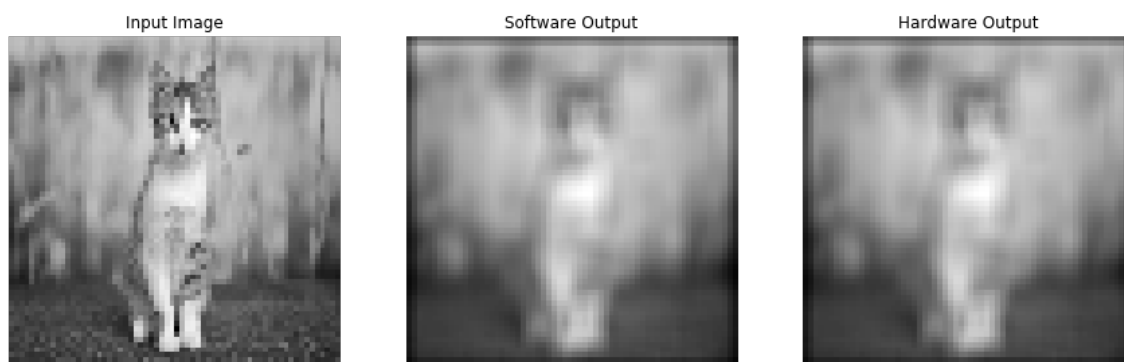
max_diff = np.max(np.abs(hw_valid - sw_valid))
print(f"Max |HW - SW| (inner window) = {max_diff:.2e}")
if max_diff < 1e-4:
    print("PASS: Hardware output matches Software golden model!")
else:
    print("WARNING: Outputs differ.")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title("Software Output")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title("Hardware Output")

for ax in axes:
    ax.axis('off')
plt.show()
```

Max |HW - SW| (inner window) = 5.72e-06

PASS: Hardware output matches Software golden model!



6. Cleanup

```
In [6]: in_buf.freebuffer()
        out_buf.freebuffer()
```

```
In [ ]:
```